

Chapter 3. Stability and Reliability

A production-ready microservice is stable and reliable. Both individual microservices and the overall microservice ecosystem are constantly changing and evolving, and any efforts made to increase the stability and reliability of a microservice go a long way toward ensuring the health and availability of the overall ecosystem. In this chapter, different ways to build and run a stable and reliable microservice are explored, including standardizing the development process, building comprehensive deployment pipelines, understanding dependencies and protecting against their failures, building stable and reliable routing and discovery, and establishing appropriate deprecation and decommissioning procedures for old or outdated microservices and/or their endpoints.

Principles of Building Stable and Reliable Microservices

Microservice architecture lends itself to fast-paced development. The freedom offered by microservices means that the ecosystem will be in a state of continuous change, never static, always evolving. Features will be added every day, new builds will be deployed multiple times per day, and old technologies will be swapped for newer and better ones at an astounding pace. This freedom and flexibility gives rise to real, tangible innovation, but comes at a great cost.

Innovation, increased developer velocity and productivity, rapid technological advancement, and the ever-changing microservice ecosystem can all very quickly be brought to a screeching halt if any piece of the microservice ecosystem becomes unstable or unreliable. In some cases, all it takes to bring the entire business down is deploying a broken build or a build containing a bug to one business-critical microservice.

A *stable* microservice is one for which development, deployment, the adoption of new technologies, and the decommissioning or deprecation of other services do not give rise to instability across the larger microservice ecosystem. This requires putting measures into place to protect

against the negative consequences that may be introduced by these types of changes. A *reliable* microservice is one that can be trusted by other microservices and by the overall ecosystem. Stability goes hand in hand with reliability because each stability requirement carries with it a reliability requirement (and vice versa): for example, stable deployment processes are accompanied by a requirement that each new deployment does not compromise the reliability of the microservice from the point of view of one of their clients or dependencies.

There are several things that can be done to ensure that a microservice is stable and reliable. A standardized *development cycle* can be implemented to protect against poor development practices. The *deployment* process can be designed so that changes to the code are forced to pass through multiple stages before being rolled out to all production servers.

Dependency failures can be protected against. Health checks, proper routing, and circuit breaking can be built into the *routing and discovery* channels to handle anomalous traffic patterns. Finally, microservices and their endpoints can be *deprecated* and/or *decommissioned* without causing any failures for other microservices.

A PRODUCTION-READY SERVICE IS STABLE AND RELIABLE

- It has a standardized development cycle.
 - Its code is thoroughly tested through lint, unit, integration, and end-to-end testing.
 - Its test, packaging, build, and release process is completely automated.
 - It has a standardized deployment pipeline, containing staging, canary, and production phases.
 - Its clients are known.
 - Its dependencies are known, and there are backups, alternatives, fallbacks, and caching in place in case of failures.
 - It has stable and reliable routing and discovery in place.
-

The Development Cycle

The stability and reliability of a microservice begins with the individual developer who is contributing code to the service. The majority of outages

changes, adding their changes to their branch, and running any unit and integration tests. This stage of development can happen anywhere: locally on a developer's laptop or on a server in a development environment. A reliable development environment—one that accurately mirrors the production world—is key, especially if testing the service in question requires making requests to other microservices or reading or writing data to a database.

Once the code has been committed to the central repository, the second step consists in having the change(s) reviewed carefully and thoroughly by other engineers on the team. If all reviewers have approved the change(s), and all lint, unit, and integration tests have passed on a new build, the change can be merged into the repository (see [Chapter 5, *Fault Tolerance and Catastrophe-Preparedness*](#), for more on lint, unit, and integration tests). Then, and only then, can the new change be introduced into the deployment pipeline.

TEST BEFORE CODE REVIEW

One way to ensure that all bugs are caught before they hit production is to run all lint, unit, integration, and end-to-end tests *before* the code review phase. This can be accomplished by having developers work on a separate branch, kicking off all tests on that branch as soon as the developer submits it for code review, and then only allowing it to reach code review (or only allowing it to be built) *after* it successfully passes all tests.

As mentioned in the section on layer 4 of the microservice ecosystem in [Chapter 1, *Microservices*](#), a lot happens in between the development cycle and the deployment pipeline. The new release needs to be packaged, built, and thoroughly tested before reaching the first stage of the deployment pipeline.

The Deployment Pipeline

There is a great deal of room for human error in microservice ecosystems, especially where deployment practices are concerned, and (as I mentioned earlier) the majority of outages in large-scale production systems are caused by bad deployments. Consider the organizational sprawl

that accompanies the adoption of microservice architecture and what it entails for the deployment process: you have, at the very least, dozens (if not hundreds or thousands) of independent, isolated teams who are deploying changes to their microservices on their own schedules, and often without cross-team coordination between clients and dependencies. If something goes wrong, if a bug is introduced into production, or if a service is temporarily unavailable during deployment, then the entire ecosystem can be negatively affected. To ensure that things go wrong with less frequency, and that any failures can be caught before being rolled out to all production servers, introducing a standardized *deployment pipeline* across the engineering organization can help ensure stability and reliability across the ecosystem.

I refer to the deployment process here as a “pipeline” because the most trustworthy deployments are those that have been required to pass a set of tests before reaching production servers. We can fit three separate stages or phases into this pipeline ([Figure 3-2](#)): first, we can test a new release in a *staging* environment; second, if it passes the staging phase, we can deploy it to a small *canary* environment, where it will serve 5%–10% of production traffic; and third, if it passes the canary phase, we can slowly roll it out to *production* servers until it has been deployed to every host.

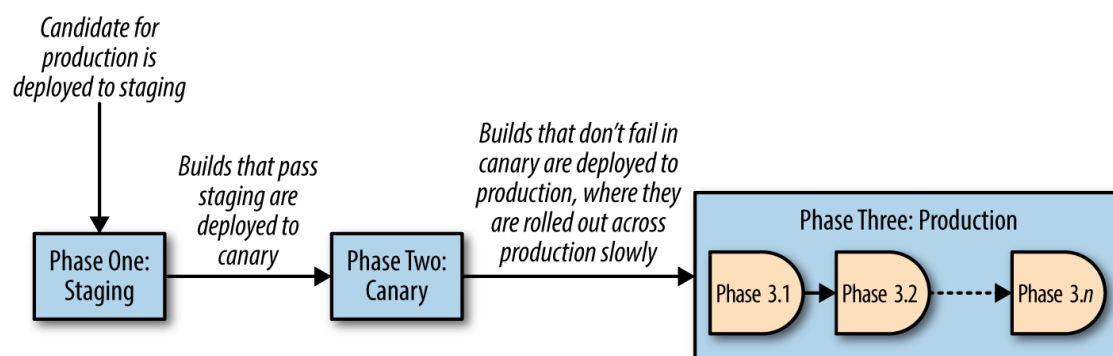


Figure 3-2. Stages of a stable and reliable deployment pipeline

Staging

Any new release can first be deployed to a *staging* environment. A staging environment should be an exact copy of the production environment: it is a reflection of the state of the real world, but without real traffic. Staging environments usually aren’t running at the same scale as production (i.e., they typically aren’t run with the same number of hosts as production, a phenomenon also known as *host parity*), because running what would

amount to two separate ecosystems can present a large hardware cost to the company. However, some engineering organizations may determine that the only way to accurately copy the production environment in a stable and reliable way is to build an identical staging environment with host parity.

For most engineering organizations, determining the hardware capacity and scale of the staging environment as a percentage of production is usually accurate enough. The necessary staging capacity can be determined by the method we will use to test the microservice within the staging phase. To test in the staging environment, we have several options: we can run mock (or recorded) traffic through the microservice; we can test it manually by hitting its endpoints and evaluating its responses; we can run automated unit, integration, and other specialized tests; or we can test each new release with any combination of these methods.

TREAT STAGING AND PRODUCTION AS SEPARATE DEPLOYMENTS OF THE SAME SERVICE

You may be tempted to run staging and production as separate services and store them in separate repositories. This *can* be done successfully, but it requires that changes be synchronized across both services and repositories, including configuration changes (which are often forgotten about). It's much easier to treat staging and production as separate “deployments” or “phases” of the same microservice.

Even though staging environments *are* testing environments, they differ from both the development phase and the development environment in that a release that has been deployed to staging is a release that is a *candidate for production*. A candidate for production must have already successfully passed lint tests, unit tests, integration tests, and code review before being deployed to a staging environment.

Deploying to a staging environment should be treated by developers with the same seriousness and caution as deploying to production. If a release is successfully deployed to staging, it can be automatically deployed to canaries, which *will* be running production traffic.

Setting up staging environments in a microservice ecosystem can be difficult, due to the complexities introduced by dependencies. If your microservice depends on nine other microservices, then it relies on those

dependencies to give accurate responses when requests are sent and reads or writes to the relevant database(s) are made. As a consequence of these complexities, the success of a staging environment hinges on the way staging is standardized across the company.

Full staging

There are several ways that the staging phase of the deployment pipeline can be configured. The first is *full staging* (Figure 3-3), where a separate staging ecosystem is running as a complete mirror copy of the entire production ecosystem (though not necessarily with host parity). Full staging still runs on the same core infrastructure as production, but there are several key differences. Staging environments of the services are, at the very least, made accessible to other services by staging-specific frontend and backend ports. Importantly, staging environments in a full-staging ecosystem communicate *only with the staging environments of other services*, and never send any requests or receive any responses from any services running in production (which means sending traffic to production ports from staging is off limits).

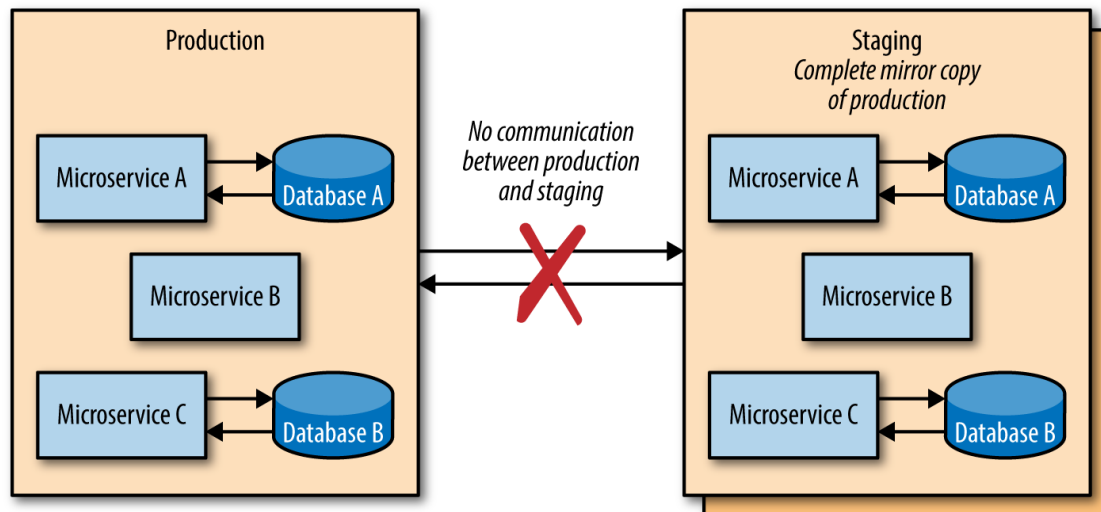


Figure 3-3. Full staging

Full staging requires every microservice to have a fully functional staging environment that other microservices can communicate with when new releases are deployed. Communicating with other microservices within the staging ecosystem can be accomplished either by writing specific tests that are kicked off when a new build is deployed to the staging environment, or as mentioned, by running old recorded production traffic or mock traffic through the service being deployed along with all upstream and downstream dependencies.

Full staging also requires careful handling of test data: staging environments should *never* have write access to any production databases, and granting read access to production databases is discouraged as well. Because full staging is designed to be a complete mirror copy of production, every microservice staging environment should contain a separate test database that it can read from and write to.

RISKS OF FULL STAGING

Caution needs to be taken when implementing and deploying full staging environments, because new releases of services will almost always be communicating with other new releases of any upstream and downstream dependencies—this may not be an accurate reflection of the real world. Engineering organizations may need to require teams to coordinate and/or schedule deployments to staging to avoid the deployment of one service breaking the staging environment for all other related services.

Partial staging

The second type of staging environment is known as *partial staging*. As the name suggests, it is not a complete mirror copy of the production environment. Rather, each microservice has its own staging environment, which is a pool of servers with (at the very least) staging-specific frontend and backend ports, and when new builds are introduced into the staging phase, they communicate with the upstream clients and downstream dependencies that are running in production ([Figure 3-4](#)).

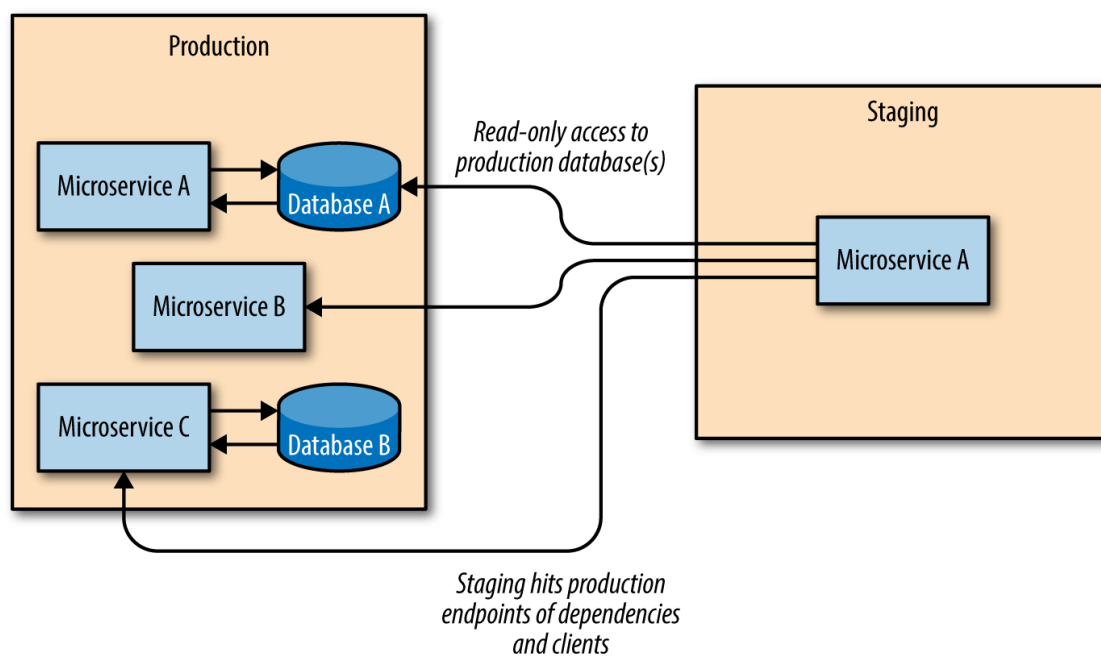


Figure 3-4. Partial staging

Partial staging deployments should hit all production endpoints of a microservice's clients and dependencies to mimic the state of the actual world as accurately as possible. Specific staging tests will need to be written and run to accomplish this, and every new feature added should probably be accompanied by at least one additional staging test to ensure that it is tested thoroughly.

RISKS OF PARTIAL STAGING

Because microservices with partial staging environments communicate with production microservices, extreme care must be taken. Even though partial staging is restricted to read-only requests, production services can easily be taken down by bad staging deploys that send bad requests and/or overload production services with too many requests.

These types of staging environments should also be restricted to read-only database access: a staging environment should never write to a production database. However, some microservices may be very write-heavy, and testing the write functionality of a new build will be essential. The most common way of doing this is to mark any data written by a staging environment as *test data* (this is known as *test tenancy*), but the safest way to do this is to write to a separate test database, since giving write access to a staging environment still runs the risk of altering real-world data. See [Table 3-1](#) for a comparison of full and partial staging environments.

Table 3-1. Full versus partial staging environments

	Full staging	Partial staging
Complete copy of production environment	Yes	No
Separate staging frontend and back-end ports	Yes	Yes
Access to production services	No	Yes
Read access to production databases	No	Yes
Write access to production databases	No	Yes
Requires automated rollbacks	No	Yes

Staging environments (full or partial) should have dashboards, monitoring, and logging just like production environments—all of which should be set up identically to the dashboards, monitoring, and logging of the production environment of the microservice (see [Chapter 6, *Monitoring*](#)). The graphs for all key metrics can be kept on the same dashboard as all production metrics, though teams may opt to have separate dashboards for each part of the deployment process: a staging dashboard, a canary dashboard, and a production dashboard. Depending on how dashboards are configured, it may be best to keep all graphs for all deployments on one dashboard and to organize them by deployment (or by metric). Regardless of how a team decides to set up their dashboards, the goal of building good and useful production-ready dashboards should not be forgotten: the dashboard(s) of a production-ready microservice should make it easy for an outsider to quickly determine the health and status of the service.

Monitoring and logging for the staging environment should be identical to the monitoring and logging of the staging and production deployments so that any failures of tests and errors in new releases that are deployed to staging will be caught before they move to the next phase of the deployment pipeline. It's extremely helpful to set up alerts and logs so that

they are differentiated and separated by deployment type, ensuring that any alerts triggered by failures or errors will specify which environment is experiencing the problem, making debugging, mitigation, and resolution of any bugs or failures rather easy and straightforward.

The purpose of a staging environment is to catch any bugs introduced by code changes before they affect production traffic. When a bug is introduced by the code, it will usually be caught in the staging environment (if it is set up correctly). Automated rollbacks of bad deploys are a necessity for partial staging environments (though are not required for full staging environments). Establishing when to revert to a previous build should be determined by various thresholds on the microservice's key metrics.

Since partial staging requires interacting with microservices running in production, bugs introduced by new releases deployed to a partial staging environment can bring down other microservices that are running in production. If there aren't any automated rollbacks in place, mitigating and resolving these problems needs to be done manually. Any steps of the deployment process that need manual intervention are points of failure not only for the microservice itself, but for the entire microservice ecosystem.

The last question a microservice team needs to answer when setting up a staging environment is how long a new release should run on staging before it can be deployed to canary (and, after that, to production). The answer to this question is determined by the staging-specific tests that are run on staging: a new build is ready to move to the next step of the deployment process as soon as all tests have passed without failing.

Canary

Once a new release has successfully been deployed to staging and passed all required tests, the build can be deployed to the next stage in the deployment pipeline: the *canary* environment. The unique name for this environment comes from a tactic used by coal miners: they'd bring canaries with them into the coal mines to monitor the levels of carbon monoxide in the air; if the canary died, they knew that the level of toxic gas in the air was high, and they'd leave the mines. Sending a new build into a canary environment serves the same purpose: deploy it to a small pool of

servers running production traffic (around 5%–10% of production capacity), and if it survives, deploy to the rest of the production servers.

CANARY TRAFFIC DISTRIBUTION

If the production service is deployed in multiple different datacenters, regions, or cloud providers, then the canary pool should contain servers in each of these in order to accurately sample production.

Since a canary environment serves production traffic, it should be considered part of production. It should have the same backend and frontend ports, and canary hosts should be chosen at random from the pool of production servers to ensure accurate sampling of production traffic. Canaries can (and should) have full access to production services: they should hit all production endpoints of upstream and downstream dependencies, and they should have both read and write access to any databases (if applicable).

As with staging, the dashboards, monitoring, and logging should be the same for canaries as for production. Alerts and logs should be differentiated and labeled as coming from the canary deployment so that developers can easily mitigate, debug, and resolve any problems.

SEPARATE PORTS FOR CANARIES AND PRODUCTION

Allocating separate frontend and backend ports for canaries and production so that traffic can be directed deliberately may seem like a good idea, but unfortunately separating out the traffic in this fashion defeats the purpose of canaries: to randomly sample production traffic on a small pool of servers to test a new release.

Automated rollbacks absolutely need to be in place for canaries: if any known errors occur, the deployment system needs to automatically revert to the last known stable version. Remember, canaries are serving production traffic, and any problems that happen are affecting the real world.

How long should a new release sit in the canary pool until developers can be satisfied that it is ready for production? This can be minutes, hours, or

even days, and the answer is determined by the microservice's traffic patterns. The traffic of every microservice is going to have some sort of pattern, no matter how strange your microservice or business may be. A new release should not leave the canary stage of deployment until a full traffic cycle has been completed. How a “traffic cycle” is defined needs to be standardized across the entire engineering organization, but the duration and requirements of the traffic cycle may need to be created on a service-by-service basis.

Production

Production is the real world. When a build has successfully made it through the development cycle, survived staging, and lived through the coal mines of the canary phase, it is ready to be rolled out to the production deployment. At this point in the deployment pipeline—the very last step—the development team should be completely confident in the new build. Any errors in the code should have been discovered, mitigated, and resolved before making it this far.

Every build that makes it to production should be completely stable and reliable. A build being deployed to production should have already been thoroughly tested, and a build should *never* be deployed to production until it has made it through the staging and canary phases without any issues. Deploying to production can be done in one fell swoop after the build has lived through the canaries, or it can be gradually rolled out in stages: developers can choose to roll out to production by percentage of hardware (e.g., first to 25% of all servers, then to 50%, then 75%, and finally 100%), or by datacenter, or by region, or by country, or any mixture of these.

Enforcing Stable and Reliable Deployment

By the time a new candidate for production has made it through the development process, has survived the staging environment, and has been deployed to the canary phase successfully, the chances of it causing a major outage are very slim, because most bugs in the code will have been caught before the candidate for production is rolled out to production. This is precisely why having a comprehensive deployment pipeline is essential for building a stable and reliable microservice.

For some developers, the delay introduced by the deployment pipeline might seem like an unnecessary burden because it delays their code changes and/or new features from being deployed straight to production minutes after they have been written. In reality, the delay introduced by the phases of the deployment pipeline is very short and easily customizable, but sticking to the standardized deployment process needs to be enforced to ensure reliability. Deploying to a microservice multiple times per day can (and does) compromise the stability and reliability of the microservice and any other services within its complex dependency chain: a microservice that is changing every few hours is rarely a stable or reliable microservice.

Developers may be tempted to skip the staging and canary phases of the deployment process and deploy a fix straight to production if, for example, a serious bug is discovered in production. While this solves the problem quickly, can *possibly* save the company from losing revenue, and can prevent dependencies from experiencing outages, allowing developers to deploy straight to production should be reserved only for the most severe outages. Without these restrictions in place, there is always the unfortunate possibility of abusing the process and deploying straight to production: for most developers, every code change, every deploy is important and may seem important enough to bypass staging and canary, compromising the stability and reliability of the entire microservice ecosystem. When failures occur, development teams should instead be encouraged to always roll back to the latest stable build of the microservice, which will bring the microservice back to a known (and reliable) state, which can run in production without any issues while the team works to discover the root cause of the failure that occurred.

HOTFIXES ARE AN ANTI-PATTERN

When a deployment pipeline is in place, there should never be any direct deployment to production unless there is an emergency, but even this should be discouraged. Bypassing the initial phases of the deployment pipeline often introduces new bugs into production, as emergency code fixes run the risk of not being properly tested. Rather than deploying a hotfix straight to production, developers should roll back to the latest stable build if possible.

Stable and reliable deployment isn't limited only to following the deployment pipeline, and there are several cases in which blocking a particular microservice from deploying can increase availability across the ecosystem.

If a service isn't meeting their SLAs (see [Chapter 2, *Production-Readiness*](#)), all deployment can be postponed if the downtime quota of the service has been used up. For example, if a service has an SLA promising 99.99% availability (allowing 4.38 minutes of downtime each month), but has been unavailable for 12 minutes in one month, then new deployments of that microservice can be blocked for the next three months, ensuring that it meets its SLA. If a service fails load testing (see [Chapter 5, *Fault Tolerance and Catastrophe-Preparedness*](#)), then deployment to production can be locked until the service is able to appropriately pass any necessary load tests. For business-critical services, whose outages would stop the company from functioning properly, it can at times be necessary to block deployment if they do not meet the production-readiness criteria established by the engineering organization.

Dependencies

The adoption of microservice architecture is sometimes driven by the idea that microservices can be built and run in isolation, as fully independent and replaceable components of a larger system. This is true in principle, but in the real world, every microservice has *dependencies*, both upstream and downstream. Every microservice will receive requests from *clients* (other microservices) that are counting on the service to perform as expected and to live up to its SLAs, as well as downstream dependencies (other services) that it will depend on to get the job done.

Building and running production-ready microservices requires developers to plan for dependency failures, to mitigate them, and to protect against them. Understanding a service's dependencies and planning for their failures is one of the most important aspects of building a stable and reliable microservice.

To understand how important this is, let's consider an example microservice called *receipt-sender*, whose SLA is four-nines (promising 99.99% availability to upstream clients). Now, *receipt-sender* depends on several

other microservices, including one called *customers* (a microservice that handles all customer information), and one called *orders* (a microservice that handles information about the orders each customer places). Both *customers* and *orders* depend on other microservices: *customers* depends on yet another microservice we'll call *customers-dependency*, and *orders* on one we'll refer to as *orders-dependency*. The chances that *customers-dependency* and *orders-dependency* have dependencies of their own are very high, so the dependency graph for *receipt-sender* quickly becomes very, very complicated.

Since *receipt-sender* wants to protect its SLA and provide 99.99% uptime to all of its clients, its team needs to make sure that the SLAs of all downstream dependencies are strictly adhered to. If the SLA of *receipt-sender* depends on *customers* being available 99.99% of the time, but the actual uptime of *customers* is only 89.99% of the time, the availability of *receipt-sender* is compromised and is now only 89.98%. Each one of the dependencies of *receipt-sender* can suffer the same hit to their availability if any of the dependencies in the dependency chain do not meet their SLAs.

A stable and reliable microservice needs to mitigate dependency failures of this sort (and yes, not meeting an SLA is a failure!). This can be accomplished by having backups, fallbacks, caching, and/or alternatives for each dependency just in case they fail.

Before dependency failures can be planned for and mitigated, the dependencies of a microservice must be known, documented, and tracked. Any dependency that could harm a microservice's SLA needs to be included in the architecture diagram and documentation of the microservice (see [Chapter 7, Documentation and Understanding](#)) and should be included on the service's dashboard(s) (see [Chapter 6, Monitoring](#)). In addition, all dependencies should be tracked by automatically creating dependency graphs for each service, which can be accomplished by implementing a distributed tracking system across all microservices in the organization.

Once all of the dependencies are known and tracked, the next step is to set up backups, alternatives, fallbacks, or caching for each dependency. The right way to do this is completely dependent on the needs of the service. For example, if the functionality of a dependency can be filled by calling the endpoint of another service, then failure of the primary dependency should be handled by the microservice so that requests are sent

to the alternative instead. If requests that need to be sent to the dependency can be held in a queue when the dependency is unavailable, then a queue should be implemented. Another way to handle dependency failures is to put caching for the dependency into place within the service: cache any relevant data so that any failures will be handled gracefully.

The type of cache most often used in these cases is a *Least Recently Used* (LRU) cache, in which relevant data is kept in a queue, and where any unused data is deleted when the cache's queue fills up. LRU caches are easy to implement (often a single line of code for each instantiation), efficient (no expensive network calls need to be made), performant (the data is immediately available), and do a decent job of mitigating any dependency failures. This is known as *defensive caching*, and it is useful for protecting a microservice against the failures of its dependencies: cache the information your microservice gets from its dependencies, and if the dependencies go down, the availability of your microservice will be unaffected. Implementing defensive caching isn't necessary for every single dependency, but if a specific dependency or set of dependencies is or are unreliable, defensive caching will prevent your microservice from being harmed.

Routing and Discovery

Another aspect of building stable and reliable microservices is to ensure that communication and interaction between microservices is itself stable and reliable, which means that layer 2 (the communication layer) of the microservice ecosystem (see [Chapter 1, *Microservices*](#)) must be built to perform in a way that protects against harmful traffic patterns and maintains trust across the ecosystem. The relevant parts of the communication layer for stability and reliability (aside from the network itself) are service discovery, service registry, and load balancing.

The *health* of a microservice at both the host level and the service level as a whole should always be known. This means that *health checks* should be running constantly so that a request is never sent to an unhealthy host or service. Running health checks on a separate channel (not used for general microservice communication) is the easiest way to ensure that health checks aren't ever compromised by something like a clogged network. Hardcoding "200 OK" responses on a */health* endpoint for health

checks isn't ideal for every microservice either, though it may be sufficient for most. Hardcoded responses don't tell you much except that the microservice was started on the host semi-successfully: any */health* endpoint of a microservice should give a useful, accurate response.

If an instance of a service on a host is unhealthy, the load balancers should no longer route traffic to it. If a microservice as a whole is unhealthy (with all health checks failing on either a certain percentage of hosts or all hosts in production), then traffic should no longer be routed to that particular microservice until the problems causing the health checks to fail are resolved.

However, health checks shouldn't be the only determining factor in whether or not a service is healthy. A large number of unhandled exceptions should also lead to a service being marked unhealthy, and *circuit breakers* should be put into place for these failures so that if a service experiences an abnormal amount of errors, no more requests will be sent to the service until the problem is resolved. The key in stable and reliable routing and discovery is this: preserve the microservice ecosystem by preventing bad actors from serving production traffic and accepting requests from other microservices.

Deprecation and Decommissioning

One often-forgotten, often-ignored cause of instability and unreliability in microservice ecosystems is the *deprecation or decommissioning* of a microservice or one of its API endpoints. When a microservice is no longer in use or is no longer supported by a development team, its decommissioning should be undertaken carefully to ensure that no clients will be compromised. The deprecation of one or more of a microservice's API endpoints is even more common: when new features are added or old ones removed, the endpoints often change, requiring that client teams are updated and any requests made to the old endpoints are switched to new endpoints (or removed entirely).

In most microservice ecosystems, deprecation and decommissioning is more of a sociological problem within the engineering organization than a technical one, making it all the more difficult to address. When a microservice is about to be decommissioned, its development team needs to

take care to alert all client services and advise them on how to accommodate the loss of their dependency. If the microservice being decommissioned is being replaced by another new microservice, or if the functionality of the microservice is being built into another existing microservice, then the team should help all clients update their microservices to send requests to the new endpoints. Deprecation of an endpoint follows a similar process: the clients must be alerted, and either given the new endpoint or advised on how to account for the loss of the endpoint entirely. In both deprecation and decommissioning, monitoring plays a critical role: endpoints will need to be monitored closely *before* the service or endpoint is completely decommissioned and/or deprecated to check for any requests that might still be sent to the outdated service or endpoint.

Conversely, failing to properly deprecate an endpoint or decommission a microservice can also have disastrous effects on the microservice ecosystem. This happens more often than developers would care to admit. In an ecosystem containing hundreds or thousands of microservices, developers are often shifted between teams, priorities change, and both microservices and technologies are swapped out for newer, better ones all of the time. When these old microservices or technologies are left to run, without any (or much) involvement, oversight, or monitoring, any failures will go unnoticed, and any failure that is noticed may not be resolved for a long period of time. If a microservice is going to be left to fend for itself, it risks compromising its clients in case of an outage—such microservices should be decommissioned rather than abandoned.

Nothing is more disruptive to a microservice than the complete loss of one of its dependencies. Nothing causes more instability and unreliability than the sudden, unexpected failure of one of its dependencies, even if the failure was planned for by another team. The importance of stable and reliable decommissioning and deprecation can honestly not be emphasized enough.

Evaluate Your Microservice

Now that you have a better understanding of stability and reliability, use the following list of questions to assess the production-readiness of your microservice(s) and microservice ecosystem. The questions are organized by topic, and correspond to the sections within this chapter.

The Development Cycle

- Does the microservice have a central repository where all code is stored?
- Do developers work in a development environment that accurately reflects the state of production (e.g., that accurately reflects the real world)?
- Are there appropriate lint, unit, integration, and end-to-end tests in place for the microservice?
- Are there code review procedures and policies in place?
- Is the test, packaging, build, and release process automated?

The Deployment Pipeline

- Does the microservice ecosystem have a standardized deployment pipeline?
- Is there a staging phase in the deployment pipeline that is either full or partial staging?
- What access does the staging environment have to production services?
- Is there a canary phase in the deployment pipeline?
- Do deployments run in the canary phase for a period of time that is long enough to catch any failures?
- Does the canary phase accurately host a random sample of production traffic?
- Are the microservice's ports the same for canary and production?
- Are deployments to production done all at the same time, or incrementally rolled out?
- Is there a procedure in place for skipping the staging and canary phases in case of an emergency?

Dependencies

- What are this microservice's dependencies?
- What are its clients?
- How does this microservice mitigate dependency failures?
- Are there backups, alternatives, fallbacks, or defensive caching for each dependency?

Routing and Discovery

- Are health checks to the microservice reliable?
- Do health checks accurately reflect the health of the microservice?
- Are health checks run on a separate channel within the communication layer?
- Are there circuit breakers in place to prevent unhealthy microservices from making requests?
- Are there circuit breakers in place to prevent production traffic from being sent to unhealthy hosts and microservices?

Deprecation and Decommissioning

- Are there procedures in place for decommissioning a microservice?
- Are there procedures in place for deprecating a microservice's API endpoints?

[Support](#) [Sign Out](#)